Министерство образования Республики Беларусь
Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Кафедра информатики

ОТЧЕТ
О ЛАБОРАТОРНОЙ РАБОТЕ № 1
УПРАВЛЕНИЕ ПРОЦЕССАМИ, ПОТОКАМИ И FIBER В WINDOWS

по дисциплине «Операционные среды и системное программирование»

Выполнил ст. гр. 453504 Л. Д. Масалова

Проверил старший препод. каф. Информатики А. Д. Сыс

Минск 2026

1 ЦЕЛЬ РАБОТЫ

Изучить механизмы управления вычислительными процессами, потоками и Fiber в операционной системе Windows и получить практические навыки использования Windows API для:

– создания и завершения процессов;
– создания и завершения потоков;
– ожидания завершения процессов и потоков;
– получения информации о состоянии процессов и потоков;
– изменения приоритетов;
– приостановки и возобновления потоков;
– создания Fiber и переключения между ними;
– измерения времени выполнения;
– исследования влияния параллельного выполнения на производительность системы.

2 ЗАДАНИЕ

Индивидуальный вариант №15: Многопоточное умножение матриц.

Разработать консольное приложение для Windows, реализующее умножение квадратных матриц A×B=C.

Требуется реализовать:

– последовательную версию;
– многопоточную версию средствами Windows API.

В многопоточной версии строки результирующей матрицы распределить между рабочими потоками. Размер матрицы N задается пользователем.

Исследовать время выполнения для количества рабочих потоков: K=1,2,4,8.

Проверить совпадение результатов двух реализаций. Каждый эксперимент выполнить не менее трех раз и рассчитать среднее время выполнения.

3 РЕЗУЛЬТАТ РАБОТЫ

3.1 Описание алгоритмов умножения матриц

В работе реализованы две версии алгоритма умножения матриц:

1 При последовательном алгоритме вычисления производятся в рамках одного первичного (главного) потока приложения. Каждый элемент результирующей матрицы рассчитывается как скалярное произведение

соответствующей строки матрицы A на столбец матрицы B. Для оптимизации обращений к оперативной памяти и максимального использования процессорного кэша L1/L2 циклы организованы в порядке i→k→j, что позволяет обращаться к памяти построчно. Результирующая матрица, полученная данным методом, принимается за эталонную.

2 При многопоточном алгоритме средствами Windows API вычисления распараллеливаются между заданным пользователем количеством рабочих потоков K ∈ {1,2,4,8}. Пространство строк результирующей матрицы C равномерно распределяется между созданными потоками по принципу полосового разбиения. Каждый рабочий поток производит вычисления независимо от остальных и сохраняет результаты строго в выделенный ему непрерывный диапазон строк матрицы C. Поскольку потоки работают с непересекающимися областями выходного массива и лишь читают данные из исходных матриц, алгоритм не требует применения блокирующих примитивов синхронизации, что исключает простои ядер процессора. Схема взаимодействия главного и рабочих потоков представлена на рисунке 1.

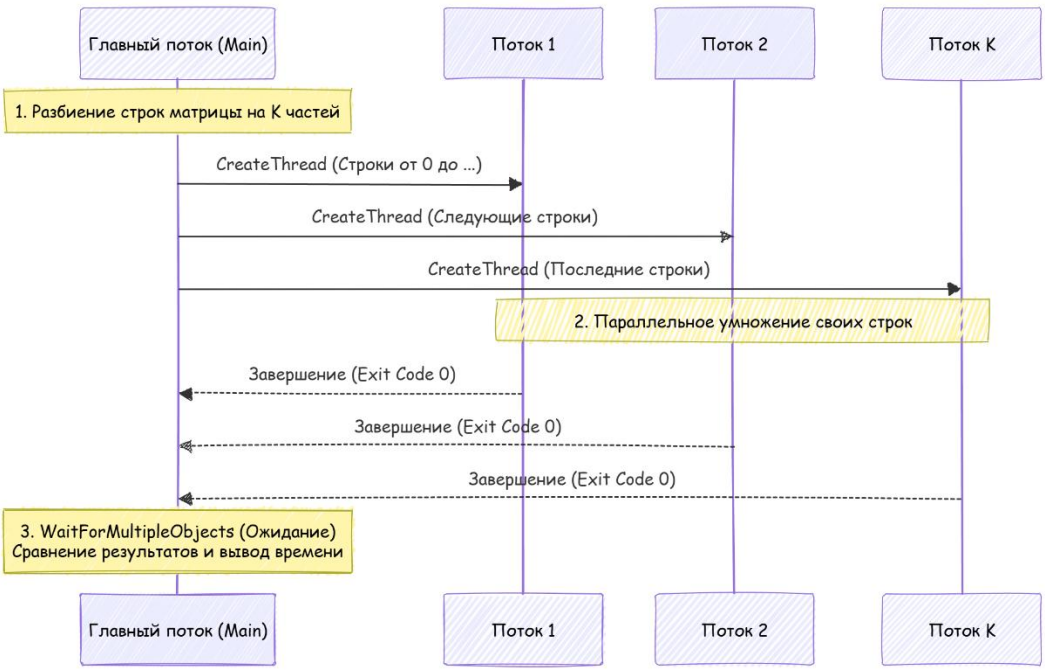


Рисунок 1 – Схема взаимодействия главного и рабочих потоков при вычислении произведения матриц

3.2 Перечень использованных функций Windows API

В ходе реализации программы были задействованы следующие системные функции Windows API:

– CreateThread() – создание и запуск рабочего потока с выделением стека и регистрацией в планировщике операционной системы;

– WaitForMultipleObjects() – барьерная синхронизация; перевод вызывающего потока в режим ожидания до момента завершения всех дескрипторов из переданного массива;

– GetExitCodeThread() – считывание кода завершения рабочего потока для подтверждения его корректного выполнения;

– SetThreadPriority() – явное назначение стандартного приоритета выполнения потоков (THREAD_PRIORITY_NORMAL);

– CloseHandle() – закрытие системных дескрипторов и удаление объектов потоков из системных таблиц ядра ОС;

– GetCurrentThreadId() – получение уникального идентификатора выполняющегося потока (TID);

– GetLastError() – запрос числового кода последней возникшей системной ошибки;

– FormatMessageA() – расшифровка числового кода ошибки в локализованное текстовое описание;

– QueryPerformanceFrequency() – определение базовой частоты аппаратного высокоточного таймера (число тиков в секунду);

– QueryPerformanceCounter() – фиксация текущего значения счетчика тактов процессора для измерения времени с субмиллисекундной точностью;

– SetConsoleCP() и SetConsoleOutputCP() – переключение кодировки консоли ввода/вывода на UTF-8 для корректной работы с кириллицей.

3.3 Основные фрагменты исходного кода

В соответствии с требованиями к реализации параллельных вычислений программный код разделен на функциональные блоки, отвечающие за обработку данных потоком, координацию их жизненного цикла и перехват системных ошибок.

Вычислительное ядро рабочего потока представлено в листинге 1. Функция принимает обобщенный указатель на структуру параметров, распаковывает адресные диапазоны строк и обращается к системному вызову GetCurrentThreadId() для фиксации системного идентификатора потока (TID). Далее в цикле производится расчет скалярных произведений для закрепленной полосы строк результирующей матрицы C, после чего функция завершается с кодом возврата 0.

```
DWORD WINAPI WorkerThread(LPVOID lpParam) {
    auto* data = (ThreadData*)lpParam;
    DWORD tid = GetCurrentThreadId();

    std::cout << "[Поток #" << data->threadNum << " | TID: " <<
tid << "] "
              << "взял строки от " << data->startRow
```

```
                << " до " << data->endRow - 1 << std::endl;

        int n = data->n;
        const Matrix& A = *(data->A);
        const Matrix& B = *(data->B);
        Matrix& C = *(data->C);

        for (int i = data->startRow; i < data->endRow; ++i) {
            for (int k = 0; k < n; ++k) {
                double r = A[i * n + k];
                for (int j = 0; j < n; ++j) {
                    C[i * n + j] += r * B[k * n + j];
                }
            }
        }
        return 0;
    }
```

Листинг 1 – Функция рабочего потока

Управление пулом параллельных потоков и их барьерная синхронизация выполняются в функции RunParallelMultiplication, приведенной в листинге 2. Главный поток производит расчет границ подмассивов с учетом возможного неделимого остатка, после чего в цикле порождает K потоков через функцию CreateThread(). Для каждого потока задается нормальный уровень приоритета THREAD_PRIORITY_NORMAL. После инициализации всех рабочих единиц главный поток переходит в состояние ожидания посредством вызова WaitForMultipleObjects(). По завершении работы всех потоков осуществляется опрос кодов завершения с помощью GetExitCodeThread() и обязательное освобождение дескрипторов функцией CloseHandle().

```
    void RunParallelMultiplication(const Matrix& A, const Matrix& B,
Matrix& C, int n, int numThreads) {
        std::fill(C.begin(), C.end(), 0.0);

        std::vector<HANDLE> threads(numThreads);
        std::vector<ThreadData> tasks(numThreads);

        int rowsPerThread = n / numThreads;
        int remainder = n % numThreads;
        int currentRow = 0;

        for (int i = 0; i < numThreads; ++i) {
            int count = rowsPerThread + (i < remainder ? 1 : 0);
```

```
            tasks[i].threadNum = i + 1;
            tasks[i].startRow = currentRow;
            tasks[i].endRow = currentRow + count;
            tasks[i].n = n;
            tasks[i].A = &A;
            tasks[i].B = &B;
            tasks[i].C = &C;

            currentRow += count;

            threads[i]   =   CreateThread(NULL,   0,   WorkerThread,
&tasks[i], 0, NULL);
            if (threads[i] == NULL) {
                PrintError("CreateThread #" + std::to_string(i + 1));
                for (int j = 0; j < i; ++j) CloseHandle(threads[j]);
                return;
            }
            SetThreadPriority(threads[i], THREAD_PRIORITY_NORMAL);
        }

        WaitForMultipleObjects(numThreads,   threads.data(),   TRUE,
INFINITE);

        for (int i = 0; i < numThreads; ++i) {
            DWORD exitCode = 0;
            GetExitCodeThread(threads[i], &exitCode);
            std::cout << "[Поток #" << (i + 1) << "] завершился с
кодом: " << exitCode << '\n';
            CloseHandle(threads[i]);
        }
    }
```

Листинг 2 – Создание, синхронизация и завершение потоков

Для обработки нештатных ситуаций и выполнения требований методички по логированию ошибок реализована функция PrintError, представленная в листинге 3. При возникновении ошибки запрашивается ее числовой код через GetLastError(), после чего системная функция FormatMessageA() транслирует его в читаемое текстовое описание на системном языке, которое выводится в поток ошибок вместе с наименованием операции.

```
    inline void PrintError(const std::string& operatName){
        auto errorCode = GetLastError();
        if(errorCode == 0) return;
```

```
        char* messageBuffer = nullptr;

        FormatMessageA(
                FORMAT_MESSAGE_ALLOCATE_BUFFER                          |
FORMAT_MESSAGE_FROM_SYSTEM,
                NULL,
                errorCode,
                0,
                (char*)&messageBuffer,
                0,
                NULL
        );

        std::cerr << "[ОШИБКА]: " << operatName
                  << "\n Код: " << errorCode
                  << "\n Описание: " << (messageBuffer ? messageBuffer
: "Неизвестно")  << '\n';

        if (messageBuffer != nullptr) {
            LocalFree(messageBuffer);
        }
    }
```

Листинг 3 – Функция обработки и вывода системных ошибок

3.4 Результаты экспериментального исследования

Экспериментальное исследование проводилось на матрицах размерности N=500 элементов (250 000 элементов в каждой матрице). В качестве базового эталона было измерено время выполнения последовательного алгоритма в рамках одного первичного потока приложения, которое составило T_последовательное = 835,87 мс.

Для оценки влияния параллелизма на производительность системы было выполнено исследование для конфигураций с количеством рабочих потоков $K \in \{1,2,4,8\}$. Согласно требованиям методики, для каждого значения K производилось по три независимых прогона, после чего рассчитывалось среднее арифметическое время выполнения и коэффициент ускорения $S = \frac{T_{\text{последовательное}}}{T_{\text{среднее}}}$. При каждом первичном запуске выполнялась автоматическая верификация корректности: поэлементное сравнение матриц подтвердило стопроцентное совпадение результатов параллельных вычислений с эталоном.

Сводные результаты экспериментальных измерений сведены в таблицу 1.

Таблица 1 – Результаты исследования производительности при различном числе потоков

Количество потоков (K)	Прогон 1, мс	Прогон 2, мс	Прогон 3, мс	Среднее время, мс	Коэфф. ускорение (S)
Последовательно	—	—	—	835,87	1,00x
K=1	893,97	918,42	897,05	903,15	0,93x
K=2	491,99	491,09	479,69	487,59	1,71x
K=4	292,43	274,09	269,72	278,75	3,00x
K=8	174,93	169,88	165,09	169,97	4,92x

Пример вывода работы программы в консоли с потоками приведен на рисунке 2.

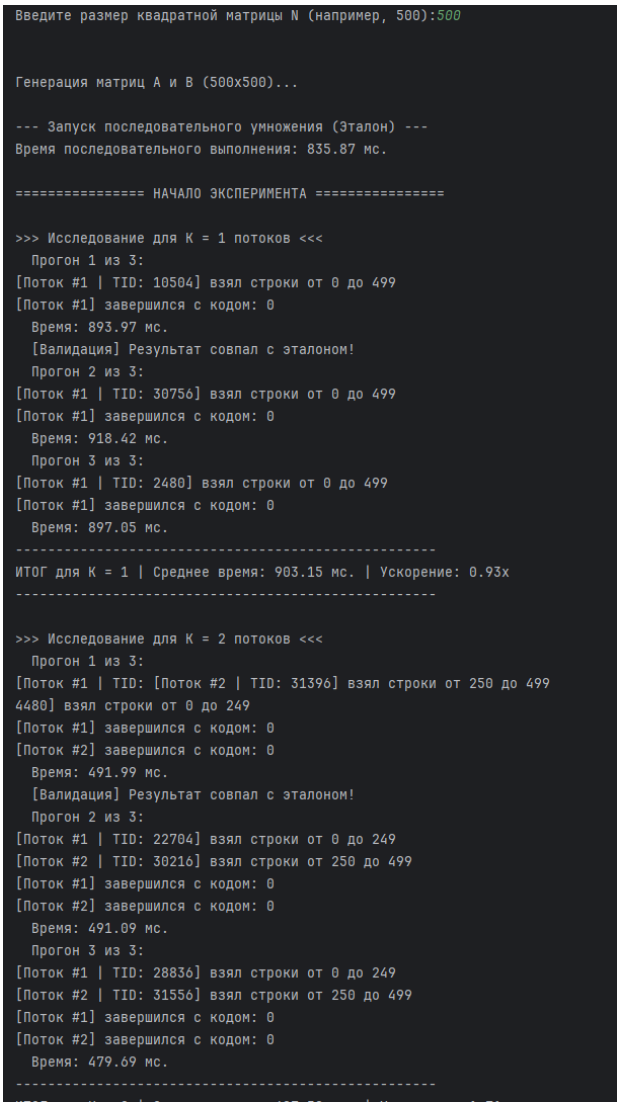


Рисунок 2 – Фрагмент вывода результатов работы программы в консоли

Анализ полученных данных показывает устойчивое сокращение времени вычислений при увеличении числа параллельных потоков от 2 до 8. При использовании одного рабочего потока через интерфейс Windows API (K=1) среднее время выполнения оказалось на 8% медленнее чисто последовательной версии (ускорение 0,93x). Данное замедление обусловлено накладными расходами операционной системы: выделением памяти под стек потока, созданием системного объекта ядра, регистрацией дескриптора, переключением контекста процессора и вызовами функций синхронизации.

При дальнейшем увеличении числа рабочих потоков до двух время вычислений сокращается почти на 42%, достигая ускорения в 1,71 раза. Конфигурация с четырьмя потоками позволяет ускорить процесс ровно в 3 раза, снижая затраченное время более чем на 66%. Наибольшая производительность была зафиксирована при использовании восьми параллельных потоков: время выполнения сократилось на 80%, что обеспечило ускорение почти в 5 раз относительно последовательной версии.

ВЫВОД

В ходе выполнения лабораторной работы были изучены механизмы низкоуровневого управления потоками в операционной системе Windows средствами интерфейса Windows API. На практике было реализовано консольное приложение, выполняющее умножение квадратных матриц в последовательном и многопоточном режимах с равномерным полосовым распределением строк между рабочими потоками.

Экспериментальное исследование наглядно подтвердило высокую эффективность параллельных вычислений: по сравнению с последовательным эталоном распараллеливание на восемь потоков позволило сократить общее время выполнения программы на 80%, обеспечив итоговое ускорение почти в 5 раз. Поэлементное сравнение матриц во всех экспериментах показало полное совпадение результатов с эталонным расчетом, что подтвердило корректность работы алгоритма и отсутствие состояний гонки при обращении к памяти.

В то же время было установлено, что увеличение количества потоков не дает строго линейного прироста скорости. При использовании двух потоков ускорение составило 1,71 раза вместо теоретических 2 раз, для четырех потоков – ровно в 3 раза вместо 4 раз, а для восьми потоков – 4,92 раза вместо 8 раз. Кроме того, создание одного системного потока через Windows API привело к замедлению алгоритма на 8% из-за затрат времени процессора на выделение стека, регистрацию дескриптора и переключение контекста планировщиком операционной системы. Непропорциональный рост скорости при дальнейшем масштабировании объясняется тем, что в программе всегда присутствуют строго последовательные участки кода (выделение памяти, расчет диапазонов строк, запуск и барьерное ожидание потоков), время выполнения которых невозможно сократить добавлением ядер, а также возрастающей аппаратной конкуренцией между потоками за общую шину оперативной памяти и кэш процессора при одновременном чтении данных.